The platform might take a few minutes to provision a load balancer for Argo CD. During the provisioning, the `EXTERNAL-IP` column will show a `<pending>` status. Wait and try again until an IP address is shown. Note it down, because we're going to use it soon.

Since the Argo CD server is now exposed via a public load balancer, we can use the external IP address to access its services. For this example, we'll use the CLI, but you can achieve the same results by opening `<argocd-external-ip>` (the IP address assigned to your Argo CD server) in a browser window. Either way, you'll have to log in with the auto-generated admin account. The username is `admin`, and the password is the one you fetched earlier. Be aware that you might get a warning, since you are not using HTTPS:

```
$ argocd login <argocd-external-ip>
```

It's now time to see continuous deployment in action with GitOps. I'll assume you have been through all the previous sections of this chapter. At this point, the commit stage of your Catalog Service repository on GitHub (`catalog-service`) should have built a container image, the acceptance stage should have triggered the Polar Deployment repository on GitHub (`polar-deployment`), and the production stage should have updated the production overlay for Catalog Service with the newest release version (polar-deployment/kubernetes/applications/catalog-service/production). Now we'll configure Argo CD to monitor the production overlay for Catalog Service and synchronize it with the production cluster whenever it detects a change in the repository. In other words, Argo CD will continuously deploy new versions of Catalog Service as made available by the deployment pipeline.